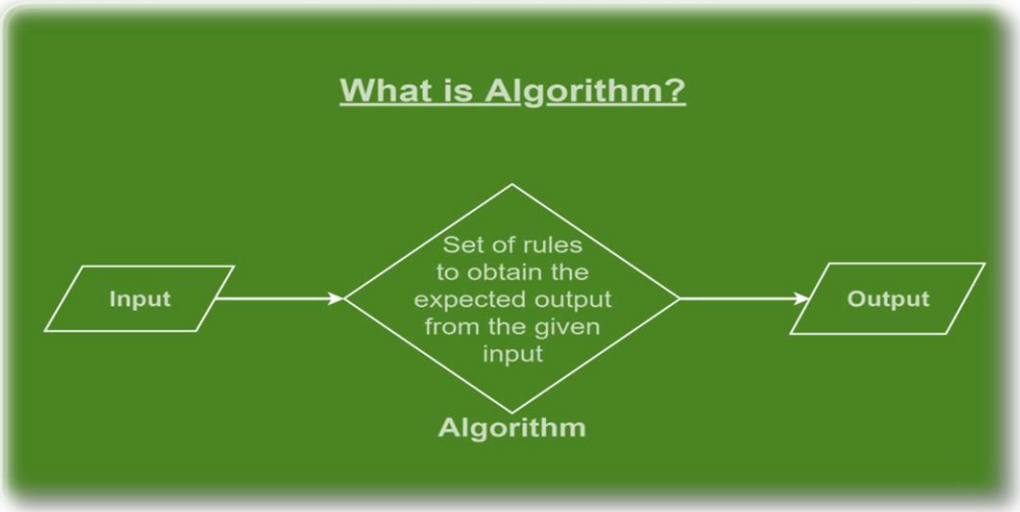
INTRODUCTION TO ALGORITHM

❖ What is Algorithm ?

- An algorithm is a **step-by-step** procedure or **set of rules** designed to perform a specific task or **solve a particular problem**.
- It is a finite sequence of **well-defined, unambiguous instructions** that, when followed, lead to a solution for a given problem or the achievement of a particular goal.
- In essence, an algorithm is a systematic way of performing a computation.



Here are some key **characteristics of algorithms**:

Well-defined:

- Each step of the algorithm must be precisely and unambiguously specified. The instructions should be clear and leave no room for interpretation.

Finite:

- An algorithm must have a finite number of steps. It cannot go on indefinitely; it must eventually halt or reach a solution after executing a certain number of steps.

Input and Output:

- An algorithm takes input, performs a series of operations, and produces an output.
- The input is the information provided to the algorithm, and the output is the result or solution generated by the algorithm.

Effectiveness:

- Algorithms are designed to be effective, meaning that they can be carried out and implemented with available resources, including time and memory.

Generality:

- Ideally, an algorithm should be applicable to a range of instances of a problem, not just a specific case.
- It should be a general solution that can handle different inputs of the same type.

Correctness:

- An algorithm should produce the correct output for all valid inputs.
- It should solve the specified problem accurately.

Optimality:

- In some cases, algorithms aim to find the best possible solution or optimize a certain criterion, such as minimizing time complexity, space complexity, or other relevant factors.

What is meant by Algorithm Analysis?

- ✓ **Algorithm analysis** is an important part of **computational complexity theory**, which provides theoretical estimation for the required resources of an algorithm to solve a specific computational problem.
- ✓ Analysis of algorithms is the determination of the amount of time and space resources required to execute it.

Why Analysis of Algorithms is Important?

- ✓ To predict the behavior of an algorithm without implementing it on a specific computer.
- ✓ It is much more convenient to have simple measures for the efficiency of an algorithm than to implement the algorithm and test the efficiency every time a certain parameter in the underlying computer system changes.
- ✓ It is impossible to predict the exact behavior of an algorithm.
- ✓ There are too many influencing factors.
- ✓ The analysis is thus only an approximation; it is not perfect.
- ✓ More importantly, by analyzing different algorithms, we can compare them to determine the best one for our purpose.

Types of Algorithm Analysis:

- **Best case**
- **Worst case**
- **Average case**

Based on the above three notations of Time Complexity there are three cases to analyze an algorithm:

Worst Case Analysis (Mostly used)

- In the **worst-case analysis**, we calculate the upper bound on the running time of an algorithm.
- We must know the case that causes a maximum number of operations to be executed.

- For Linear Search, the worst case happens when the element to be searched **(x)** is not present in the array.
- When x is not present, the **search()** function compares it with all the elements of **arr[]** one by one.
- Therefore, the worst-case time complexity of the linear search would be **O(n).**

Best Case Analysis (Very Rarely used)

- In **the best-case analysis**, we calculate the lower bound on the running time of an algorithm.
- We must know the case that causes a minimum number of operations to be executed.
- In the linear search problem, the best case occurs when **x** is present at the first location.
- The number of operations in the best case is constant (**not dependent on n**). So time complexity in the best case would be.

Average Case Analysis (Rarely used)

- In **average case analysis**, we take all possible inputs and calculate the computing time for all of the inputs.
- Sum all the calculated values and divide the sum by the total number of inputs.
- We must know (or predict) the distribution of cases.
- For the linear search problem, let us assume that all cases are uniformly distributed (including the case of x not being present in the array).
- So we sum all the cases and divide the sum by **(n+1).**

Following is the value of average-case time complexity.

$$\text{Average Case Time} = \sum_{i=1}^{n}\frac{\theta (i)}{(n+1)} = \frac{\theta (\frac{(n+1)*(n+2)}{2})}{(n+1)} = \theta (n)$$

Need of Algorithm

- ✓ To understand the basic idea of the problem.
- ✓ To find an approach to solve the problem.
- ✓ To improve the efficiency of existing techniques.
- ✓ To understand the basic principles of designing the algorithms.
- ✓ To compare the performance of the algorithm with respect to other techniques.
- ✓ It is the best method of description without describing the implementation detail.
- ✓ The Algorithm gives a clear description of requirements and goal of the problem to the designer.
- ✓ A good design can produce a good solution.
- ✓ To understand the flow of the problem.
- ✓ 12. With the help of algorithm, we convert art into a science.
- ✓ 13. To understand the principle of designing.

Advantages of Algorithms:

- It is easy to understand.
- An algorithm is a step-wise representation of a solution to a given problem.
- In an Algorithm the problem is broken down into smaller pieces or steps hence, it is easier for the programmer to convert it into an actual program.

Disadvantages of Algorithms:

- Writing an algorithm takes a long time so it is time-consuming.
- Understanding complex logic through algorithms can be very difficult.
- Branching and Looping statements are difficult to show in Algorithms(imp).

Properties of Algorithm:

- It should terminate after a finite time.
- It should produce at least one output.
- It should take zero or more input.
- It should be deterministic means giving the same output for the same input case.
- Every step in the algorithm must be effective i.e. every step should do some work.

Application of Algorithm:

Here are some key applications of algorithms in DAA:

➢ **Sorting and Searching**:

- Algorithms are extensively used for **sorting and searching data**.
- Sorting algorithms like **quicksort, mergesort, and heapsort** are applied to arrange data in a specific order.
- Searching algorithms like binary search efficiently locate items in a sorted collection.

➢ **Graph Algorithms:**

- Algorithms are fundamental in solving problems related to graphs.
- Graph algorithms, such as **Dijkstra's algorithm for shortest paths, Kruskal's algorithm for minimum spanning trees, and depth-first search**, are widely used in **network optimization, routing, and connectivity analysis.**

➢ **Dynamic Programming:**

- **Dynamic programming** is a technique that involves breaking down a problem into smaller subproblems and solving each subproblem only once, storing the solutions to subproblems in a table to avoid redundant computations.

- Algorithms based on dynamic programming are employed in various optimization problems, such as the **knapsack problem** and **shortest path problems.**

➢ Greedy Algorithms:

- **Greedy algorithms** make locally optimal choices at each stage with the hope of finding a global optimum.
- These algorithms are used in problems like Huffman coding for data compression, scheduling, and certain optimization problems.

➢ Divide and Conquer:

- **Divide and conquer** is a technique where a problem is broken down into smaller subproblems, solved independently, and then combined to find the solution to the original problem.
- Algorithms like merge sort and quicksort use this approach for sorting, and algorithms like the **Fast Fourier Transform (FFT)** use it for signal processing.

➢ String Matching Algorithms:

- Algorithms for **string matching** are essential in text processing and pattern recognition.
- The **Knuth-Morris-Pratt algorithm**, **Boyer-Moore algorithm**, and **Rabin-Karp algorithm** are examples of algorithms used in string matching.